

Ciencia de Datos para Politólogos:

de la pregunta al modelo, del modelo a la decisión

Tomás Olego

2026-04-21

Carrera de Ciencia Política - Universidad de Buenos Aires

Ruta y ritmo de la clase

Bloque	Tiempo	Pregunta organizadora
Apertura e idea central	15 min	¿Qué problema resuelve realmente data science?
Caso de viviendas y formulación	20 min	¿Qué significa predecir “precio” y para qué?
Evaluación y generalización	45 min	¿Cómo evitamos engañarnos con buen ajuste pasado?
Comparación de modelos	35 min	¿Qué compromisos aceptamos al modelar?
Del modelo al sistema real	30 min	¿Cómo se sostiene una solución en operación?
Streaming y drift	15 min	¿Qué cambia cuando los datos llegan en flujo?
Mapa general y cierre	20 min	¿Dónde se ubica este caso dentro de ML?

Decisiones apoyadas en datos

¿Cómo pasamos de una pregunta sobre el mundo a una decisión confiable apoyada en datos?

Tesis

Data science **no** es una colección de algoritmos. Es un **proceso de decisión** que organiza datos, incertidumbre, criterios de evaluación y consecuencias de uso.

Hilo conductor

Seguiremos un único caso:

$$x \longmapsto \hat{y}$$

donde x resume las características observables de una vivienda y $\hat{y}$ su precio esperado. Cada concepto aparecerá porque resuelve un problema real del recorrido.

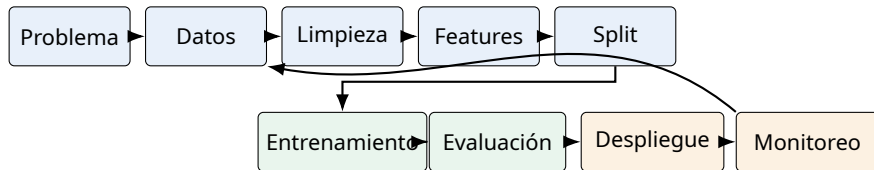
Describir, inferir, predecir y operar no son lo mismo

Modo	Pregunta	Resultado	Ejemplo en viviendas
Descriptivo	¿Qué está pasando?	resúmenes y patrones	precio medio por barrio
Inferencial	¿Qué evidencia hay sobre una relación?	estimación y contraste	efecto de la cochera sobre el precio
Predictivo	¿Qué pasará para un caso nuevo?	regla de predicción	estimar el precio de una vivienda no vista
Operativo	¿Cómo integramos eso en una decisión?	proceso y monitoreo	usar el modelo en una tasación real

Importante

ML trabaja sobre todo en lo predictivo; data science organiza el recorrido completo.

El verdadero objeto de trabajo es el pipeline completo



Idea clave: el algoritmo ocupa sólo una parte del sistema; el monitoreo puede obligarnos a revisar la formulación inicial.

Caso central

Predicción del precio de viviendas: un mismo caso para introducir formulación, evaluación, comparación de modelos y operación.

El caso de viviendas nos sirve como ancla conceptual

Pregunta del caso

Dadas las características observables de una vivienda, ¿qué precio esperaríamos observar en el mercado?

Por qué funciona bien como caso docente

- es intuitivo y cercano;
- tiene un target numérico claro;
- hace visible la diferencia entre modelos;
- permite discutir despliegue y monitoreo.

Uso potencial

- tasación inicial;
- alerta de sub/sobrepresco;
- apoyo a decisiones comerciales.

La primera decisión no es el modelo: es definir bien la predicción

Formulación	Cuándo se predice	Dato disponible	Uso
Precio de publicación	antes de publicar	atributos declarados del inmueble	salida inicial al mercado
Precio de cierre Precio esperado a 30 días	después de negociación hoy, para seguimiento	más señales del proceso atributos + contexto temporal	valuación ex post ajuste comercial
Rango de tasación	en la visita o análisis	información incompleta	apoyo a tasación

Importante

Cambiar el instante de decisión cambia target, features válidas, métrica y leakage.

Del caso concreto a la formulación predictiva

Representación

Cada vivienda se resume en un vector

$$\mathbf{x} = (x_1, \dots, x_p) \in X, \quad y \in Y \subseteq \mathbb{R}.$$

En el caso de viviendas:

$$\mathbf{x} = (m^2, \text{ambientes}, \text{barrio}, \text{antigüedad}, \dots),$$
$$y = \text{precio}.$$

Objetivo

Aprender una función

$$f : X \rightarrow Y, \quad \hat{y} = f(\mathbf{x}),$$

que funcione para viviendas no vistas,
no sólo para las observadas.

Generalizar significa que la regla sigue siendo útil cuando aparece una vivienda nueva.

Qué significa “modelo” en esta clase

Idea central

En esta clase, un **modelo** es una **regla de predicción**

$$\hat{y} = f(\mathbf{x}),$$

que toma información observable $\mathbf{x}$ y devuelve una salida $\hat{y}$. Pero, al empezar, no conocemos la regla exacta. Lo que elegimos primero es una **familia de reglas posibles**

$$\mathcal{F}_h = \{f : X \rightarrow Y\},$$

donde h resume decisiones tomadas **antes** de entrenar.

En viviendas

$\mathbf{x} = (m^2, \text{ambientes, barrio, antigüedad, } \dots)$

y el modelo produce una predicción de precio

$$\hat{y} = f(\mathbf{x}).$$

Qué fija h

El símbolo h nos recuerda que, antes de ajustar con datos, todavía debemos decidir **qué tan simple o flexible** permitiremos que sea esa familia.

Con los datos históricos y con h fijado, el entrenamiento elige una regla ajustada $\hat{f}_h \in \mathcal{F}_h$.

Hiperparámetros, complejidad y cantidad de datos

Dónde aparecen los hiperparámetros

Las decisiones resumidas en h se llaman **hiperparámetros**. No describen una vivienda; describen **la forma y la complejidad permitida** para la familia $\mathcal{F}_h$.

h más restrictivo

La familia es más simple:

- representa menos patrones;
- suele necesitar menos datos;
- puede quedarse corta si la relación real es compleja.

h más flexible

La familia es más amplia:

- puede capturar patrones más finos;
- también puede ajustar ruido;
- suele exigir más datos y mejor validación.

Importante

La complejidad no se juzga sola: se juzga junto con la cantidad de datos y con la capacidad de generalizar a casos nuevos.

La función de pérdida convierte error en criterio de aprendizaje

Definición

La **función de pérdida** mide cuánto cuesta predecir $\hat{y}$ cuando el valor real es y :

$$\ell(y, \hat{y}).$$

Con datos de entrenamiento $(\mathbf{x}_i, y_i)_{i=1}^n$, el ajuste busca minimizar la pérdida promedio:

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(\mathbf{x}_i)).$$

Importante

La pérdida fija **cómo aprende** el modelo en train; luego validation y test nos permiten preguntar si ese criterio de ajuste realmente **generaliza** y si está alineado con la **decisión** que queremos tomar.

Lectura en viviendas

Si usamos pérdida cuadrática,

$$\ell(y, \hat{y}) = (y - \hat{y})^2,$$

los errores grandes pesan mucho más. Si usamos pérdida absoluta,

$$\ell(y, \hat{y}) = |y - \hat{y}|,$$

cada desvío cuenta de manera lineal.

Elegir una métrica también es decidir qué errores importan

En viviendas

No todos los errores se sienten igual:

- errar USD 5,000 puede ser tolerable;
- errar USD 100,000 puede cambiar una decisión comercial.

Consecuencia

La métrica no es neutra:

métrica $\Rightarrow$ preferencia sobre el error.

Elegirla es hacer explícito qué discrepancias castigamos más.

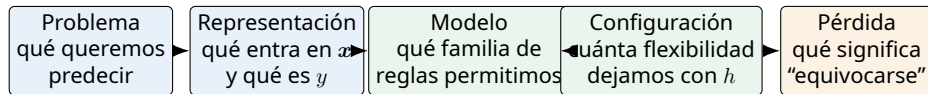
MAE, MSE, RMSE y R^2 : cuatro lecturas del desempeño

Métrica	Definición	Lectura
MAE	$\frac{1}{n} \sum_{i=1}^n y_i - \hat{y}_i $	error medio en las mismas unidades del precio
MSE	$\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$	castiga con fuerza los errores grandes
RMSE	$\sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$	mantiene esa penalización, pero vuelve a la escala del precio
R^2	$1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$	compara al modelo con la referencia "predecir siempre el promedio"

Cuidado

Un R^2 alto no garantiza utilidad operativa; no reemplaza a MAE o RMSE.

Qué ya quedó definido antes de entrenar



Importante

Entrenar no empieza desde cero: empieza **después** de haber definido problema, variables, familia de modelos, nivel de complejidad y criterio de error. Entonces sí buscamos, con datos observados, una regla ajustada y luego preguntamos si **generaliza**.

Entrenar es minimizar la pérdida sobre los datos observados

Idea formal

Una vez fijados el problema, la familia de modelos $\mathcal{F}_h$ y la pérdida ℓ , entrenar consiste en elegir, dentro de $\mathcal{F}_h$, la regla que minimiza la pérdida empírica:

$$\hat{f} \in \arg \min_{f \in \mathcal{F}_h} \hat{R}_n(f) = \arg \min_{f \in \mathcal{F}_h} \frac{1}{n} \sum_{i=1}^n \ell(y_i, f(\mathbf{x}_i)).$$

Lectura intuitiva

Con las viviendas observadas, buscamos la regla cuyas predicciones reciben el **menor castigo total** según el criterio elegido.

Si cambiamos la pérdida, también cambia qué significa **ajustar bien**.

Importante

Entrenar responde una pregunta **interna a train**: qué regla ajusta mejor los datos vistos. La pregunta siguiente será más exigente: **¿esa regla también funciona con viviendas nuevas?**

Evaluar es estimar generalización

La evaluación no es un apéndice técnico: es el centro del razonamiento predictivo.

Entrenar no es lo mismo que generalizar

Entrenar

Ajustar parámetros usando ejemplos observados:

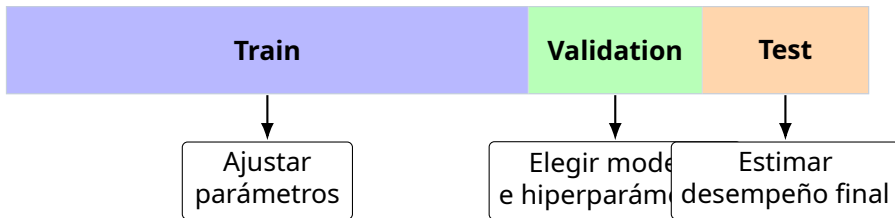
$(x_1, y_1), \dots, (x_n, y_n) \Rightarrow$ parámetros ajustados.

Generalizar

Conservar buen desempeño cuando llegan viviendas nuevas, que no participaron del ajuste.

La pregunta correcta no es *qué tan bien ajusta el pasado*, sino *qué tan bien generaliza fuera de muestra*.

Train, validation y test simulan el futuro con datos del pasado



Separamos **tres decisiones distintas**: aprender, comparar y reportar una evaluación honesta.

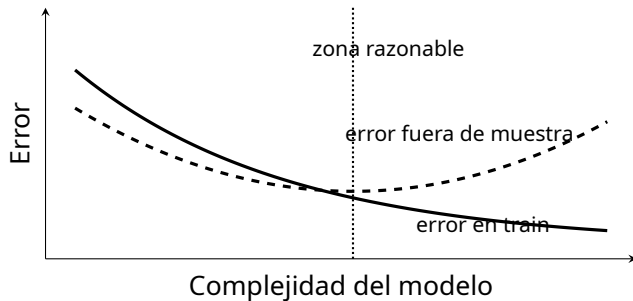
Cada partición existe para proteger una decisión distinta

Conjunto	Se usa para	No debe usarse para
Train	ajustar parámetros y aprender transformaciones	reportar desempeño final
Validation	comparar modelos, hiperparámetros o umbrales	seguir afinando hasta memorizar ese conjunto
Test	estimar generalización final	tomar decisiones repetidas de tuning

Principio

El conjunto de test vale precisamente porque se mantiene reservado.

Más complejidad puede bajar el error en train y empeorar el error real



Sobreajuste: mejorar el ajuste interno deja de mejorar la capacidad de generalizar.

La pregunta central no es cuánto ajusta, sino cuánto generaliza

Secuencia conceptual del caso

Formulamos el problema, fijamos X e Y , entrenamos una función f , separamos train/validation/test, evitamos leakage y elegimos métricas coherentes.

Todo eso existe para responder una sola pregunta:
¿cómo esperamos que el modelo funcione en viviendas no vistas?

Con esta lógica instalada, recién ahora conviene comparar modelos.

Sesgo, varianza y cantidad de datos

Lectura conceptual

Bajo pérdida cuadrática, de manera esquemática, el error de generalización puede pensarse como

$$\mathbb{E}[(Y - \hat{y})^2] \approx \text{sesgo}^2 + \text{varianza} + \text{ruido}.$$

Sesgo: error por imponer una estructura demasiado rígida.

Varianza: sensibilidad excesiva a la muestra concreta con la que entrenamos.

Consecuencia práctica

Modelos más simples	Modelos flexibles	más
más sesgo menos varianza piden menos datos	menos sesgo más varianza suelen pedir más da- tos	

Puente hacia la comparación de modelos

En viviendas, una regla demasiado rígida puede perder regularidades importantes; una demasiado flexible puede ajustarse en exceso a la muestra observada. La comparación entre modelos buscará precisamente entender **qué equilibrio** propone cada uno entre estructura, flexibilidad y capacidad de generalizar.

Comparar modelos

No se trata de decidir cuál “gana” en abstracto, sino qué compromisos aceptamos para generalizar mejor.

Linear Regression: claridad e interpretabilidad, con una forma rígida

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Lectura

Una vez fijadas las demás variables, cada coeficiente describe cómo cambia el precio esperado al variar una feature.

En viviendas

- más m^2 suele empujar el precio hacia arriba;
- más antigüedad puede empujarlo hacia abajo;
- barrio y cochera entran como covariables explicativas.

Rol pedagógico

Es una **baseline natural**: simple, rápida y fácil de interpretar.

Ridge Regression: conservar la forma lineal, pero ganar estabilidad

$$\min_{\beta_0, \beta} \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

Intuición

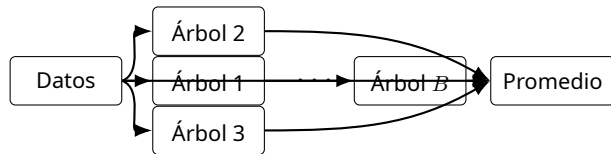
La penalización L^2 empuja los coeficientes hacia tamaños menores y reduce inestabilidad cuando hay variables correlacionadas.

En viviendas

- m^2 y ambientes suelen correlacionarse;
- expensas, barrio y distancia al centro pueden superponerse parcialmente.

$\lambda \uparrow \Rightarrow$ más restricción $\lambda \downarrow \Rightarrow$ menos restricción

Random Forest: perder transparencia para ganar flexibilidad predictiva



Qué gana

- no linealidades;
- interacciones automáticas;
- buena performance inicial sin demasiado tuning.

En viviendas

Puede capturar que m^2 valga distinto según el barrio y que la cochera no pese igual en toda zona.

Tres modelos, una misma tarea: tres maneras de intentar generalizar

Modelo	Qué gana	Qué sacrifica	Cuándo suele servir
Linear Regression	interpretabilidad, simplicidad, baseline sólida	subajuste ante relaciones complejas	cuando importa la transparencia
Ridge Regression	más estabilidad ante colinealidad	sigue siendo lineal	cuando hay muchas variables correlacionadas
Random Forest	flexibilidad, interacción y no linealidad	menos transparencia y más costo operativo	cuando prima la performance predictiva

La comparación correcta no es entre nombres de modelos, sino entre **claridad, estabilidad, flexibilidad y costo**.

¿Qué compromisos estoy dispuesto a aceptar
entre claridad, flexibilidad, estabilidad y costo operacional?

Criterios para decidir

- estructura esperable del problema;
- calidad y cantidad de datos;
- necesidad de generalización;
- exigencia de explicación o auditoría.

Pregunta docente

Si un banco exigiera justificar cada tasación, ¿seguiría siendo una buena elección un modelo muy opaco aunque mejore levemente el RMSE?

Preparar datos también es modelar

Operaciones típicas

- limpieza e imputación;
- joins;
- encoding;
- escalado;
- feature engineering.

Ejemplos

Viviendas: codificar barrio, construir distancia al centro, combinar antigüedad y expensas.

Logs o sensores: agregar por ventana temporal o resumir actividad reciente.

Lectura conceptual

Preparar no es “ordenar columnas”: es decidir qué información será visible para el modelo y bajo qué representación.

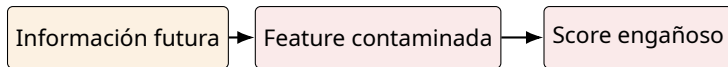
Leakage: la manera más rápida de fabricar una ilusión de performance

Definición

Hay **leakage** cuando el modelo usa información que no estaría disponible al momento real de predecir, o cuando el procedimiento deja filtrar información entre conjuntos.

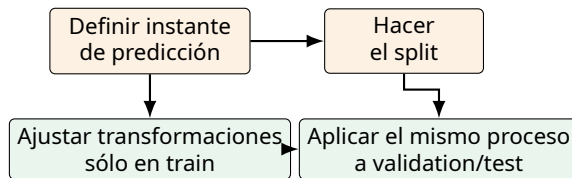
Ejemplo en viviendas

Queremos predecir **precio de publicación**, pero construimos una feature usando **precio de cierre**. Ese dato todavía no existe cuando publicamos.



performance aparente es mucho mayor que la performance real

Prevenir leakage exige tratar todo el pipeline como parte del aprendizaje



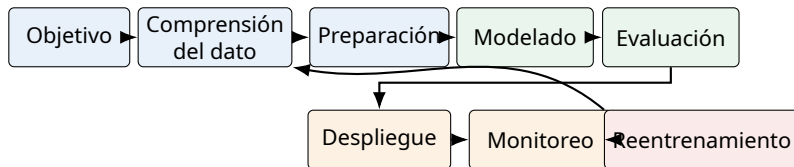
Regla práctica

Todo paso que aprende algo de los datos —imputación, escalado, selección de variables, tuning— debe respetar la separación entre conjuntos.

Del modelo al sistema real

Elegir una función predictiva no cierra el trabajo: recién ahí empieza su vida operativa.

El modelo vive dentro de un workflow real



Mensaje: un modelo útil no termina cuando se entrena; recién ahí empieza su comportamiento observable como sistema.

Antes del algoritmo se decide gran parte del éxito del proyecto

Traducir el problema en un objetivo operable

- ¿Qué queremos predecir exactamente?
- ¿Para quién?
- ¿Qué acción seguirá a la predicción?
- ¿Cuál es el costo de errar?

Entender el dato

- estructura y granularidad;
- faltantes y sesgos;
- cobertura temporal;
- cambios de captura o definición.

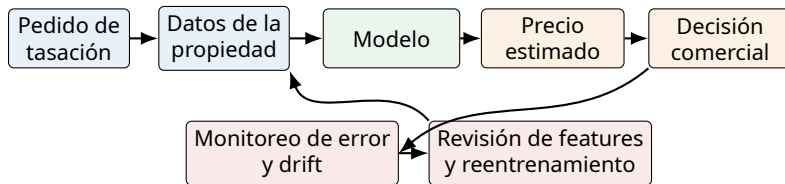
Muchos proyectos fallan no en el algoritmo, sino en una formulación deficiente o en datos mal comprendidos.

Evaluación, despliegue y monitoreo forman una sola cadena operativa

Etapa	Pregunta	Qué importa
Evaluación	¿El desempeño está alineado con la decisión?	métrica, validación, comparación honesta
Despliegue	¿La predicción llega a tiempo y en el formato correcto?	batch/tiempo real, integración, consistencia de entrada
Monitoreo	¿El sistema sigue funcionando cuando el mundo cambia?	drift, latencia, alarmas, reentrenamiento

Evaluar no es cerrar el notebook: es decidir si el sistema merece ser usado y sostenido.

El caso de viviendas reaparece como sistema completo



La salida del modelo no es el final de la historia: entra en una decisión real, con seguimiento y corrección.

Cuando los datos llegan en movimiento

Streaming cambia la pregunta: el dataset deja de estar cerrado y la adaptación pasa a ser parte del problema.

Batch y streaming responden a restricciones distintas

Aspecto	Batch	Streaming
Cómo llegan los datos	por lotes acumulados	como flujo continuo de eventos
Tiempo de respuesta	moderado o diferido	bajo, con presión de latencia
Unidad típica	dataset relativamente cerrado	ventanas temporales o eventos sucesivos
Ejemplos	scoring nocturno, reporte diario	fraude en tiempo real, sensores, clics
Desafío principal	ciclos estables	responder mientras el entorno cambia

Cuando los datos no esperan, la operación del modelo pasa al centro del problema.

Drift: cuando el mundo deja de parecerse al que vio el modelo

Dos cambios posibles

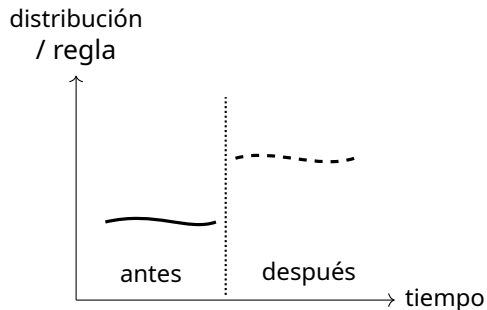
- **Data drift:** cambia la distribución de las entradas.
- **Concept drift:** cambia la relación entre entradas y target.

Respuesta posible

Monitoreo, reentrenamiento periódico u **online learning**.

Ejemplo

En una plataforma inmobiliaria, cambios de oferta o del mercado pueden volver obsoleto un tasador entrenado hace meses.



Mapa general del aprendizaje automático

Nuestro caso fue supervisado; ahora lo ubicamos dentro del panorama más amplio.

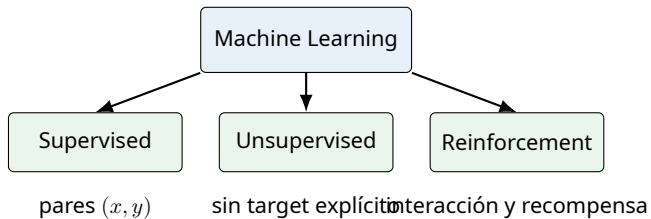
Desde el caso de viviendas hacia el mapa general

Puente conceptual

Durante casi toda la clase trabajamos con pares

$$(\mathbf{x}_i, y_i), \quad i = 1, \dots, n,$$

y aprendimos una regla para predecir y a partir de x .



La pregunta ya no es sólo qué modelo usar, sino qué información tenemos y qué feedback recibe el sistema.

Tres paradigmas, tres estructuras de información

Paradigma	Qué observamos	Pregunta principal	Ejemplo
Supervised learning	pares (x, y)	predecir un target nuevo	precio, churn, fraude
Unsupervised learning	datos x sin target explícito	hallar estructura o anomalías	segmentación, clusters
Reinforcement learning	estados, acciones y recompensas	aprender una política secuencial	control, robótica, recomendación

Lectura útil

La diferencia no está sólo en el algoritmo, sino en la información disponible y en cómo llega el feedback.

Rescapitulación: seis respuestas y una misma idea

- **¿Qué problema resuelve data science?**

Transformar datos en decisiones confiables bajo incertidumbre.

- **¿Cuál es el flujo típico?**

Problema, datos, preparación, modelado, evaluación, despliegue y monitoreo.

- **¿Qué es entrenar, validar y testear?**

Ajustar, comparar con honestidad y estimar generalización final.

- **¿Por qué cambian los resultados entre modelos?**

Porque cada modelo acepta compromisos distintos entre interpretabilidad, estabilidad y flexibilidad.

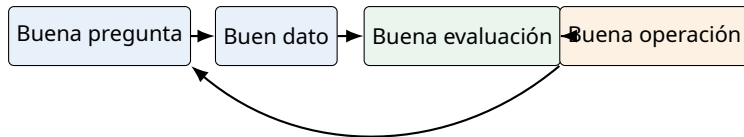
- **¿Qué cambia con streaming?**

Los datos llegan continuamente, el entorno cambia y el sistema debe adaptarse.

- **¿Dónde ubicar supervised, unsupervised y RL?**

Como tres formas distintas de aprender según la estructura del feedback.

Un modelo excelente para una pregunta mal formulada sigue siendo una mala solución.



Data science opera bien cuando conecta criterio estadístico, diseño del problema y responsabilidad operativa.

Libros recomendados para seguir profundizando

- **Python Data Science Handbook** — Jake VanderPlas
Base práctica de data science en Python: NumPy, pandas, visualización y primeros modelos.
Dificultad: baja
- **Introduction to Machine Learning with Python** — Andreas C. Müller, Sarah Guido
Muy buena puerta de entrada a machine learning aplicado con `scikit-learn`.
Dificultad: baja-media
- **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow** — Aurélien Géron
Texto más amplio y exigente, con ML clásico y deep learning.
Dificultad: media-alta
- **The Elements of Statistical Learning** — Hastie, Tibshirani, Friedman
Referencia clásica, más teórica y matemáticamente más demandante.
Dificultad: alta
- **An Introduction to Statistical Learning (with Applications in Python)** — James, Witten, Hastie, Tibshirani, Taylor
Versión más pedagógica, accesible y orientada a implementación que *The Elements of Statistical Learning*.
Dificultad: media
- **Designing Machine Learning Systems** — Chip Huyen
Menos enfocado en aprender algoritmos desde cero y más en diseñar sistemas de ML listos para producción.
Dificultad: media

Nivel de dificultad

El más difícil de los seis es **The Elements of Statistical Learning**. Luego siguen **Hands-On Machine Learning** y, en otro registro, **Designing Machine Learning Systems**, que exige más criterio de sistema y producción que matemática. Los más accesibles para empezar suelen ser **Python Data Science Handbook** e **Introduction to Machine Learning with Python**.